

Listas (list)

Clase 5

Algunas palabras claves

pass, **continue** y **break** se usan para controlar el flujo de ejecución del programa.

- **pass**: Se utiliza cuando se necesita una declaración vacía en un bloque de código. En otras palabras, no hace nada y simplemente actúa como marcador de posición.
 - **continue**: Se utiliza en bucles (como while y for) para saltar a la siguiente iteración del bucle sin ejecutar el código restante en el bloque actual.
 - **break**: Se utiliza para salir de un bucle (como while y for) antes de que se complete todas las iteraciones.
- 

Ejemplo

```
def funcion_a_implementar():  
    pass
```

```
for i in range(10):  
    if i == 5:  
        continue  
    print(i, end=' ')
```

```
print(i, end='\n\n')
```

```
while i > 0:  
    if i == 5:  
        break  
    i -= 1  
    print(i, end='\n\n')
```



Colecciones - Listas

Una lista es una colección ordenada de elementos. Las listas son estructuras heterogéneas, es decir que pueden contener cualquier tipo de datos, inclusive listas.

```
variedad = [1, "dos", [3, "cuatro"], True]
```



Acceso a los elementos

```
variedad = [1, "dos", [3, "cuatro"], True]
```

Los elementos se separan por comas (,) y se acceden de la siguiente forma:

```
print(variedad[3]) # True
```

```
print(variedad[2][1]) # "cuatro"
```

```
print(variedad[1:-1]) # ["dos", [3, "cuatro"]]
```



Mutable

Las listas son objetos mutables ya que permiten alterar sus elementos luego de haber sido creadas.

```
variedad = [1, "dos", [3, "cuatro"], True]
```

```
variedad[3] = "cambio"
```

```
print(variedad) # [1, "dos", [3, "cuatro"], "cambio"]
```



Manejo de listas

- Diferencia entre hacer `lis2 = lis1` y `lis2 = lis1[:]`
 - `lis2 = lis1` hace que el `lis2` apunte a `lis1` (ambos apuntan a la misma zona de memoria)
 - `lis2 = lis[:]` hace que dos direcciones de memoria tengan el mismo contenido
- Uso de método `copy` en Python 3.
 - `lis2 = lis.copy()`



Operadores

```
lis1 = [ 22, True, 'una lista', [1,7] ]
```

- **+** : Para concatenar listas

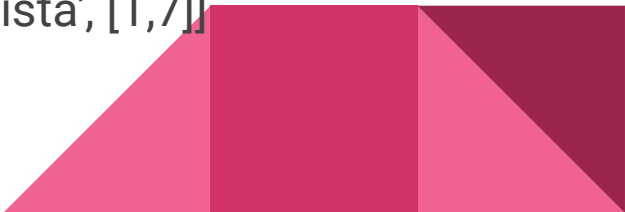
```
lis2 = lis1 + [9,8,7]
```

lis2 quedaría: [22, True, 'una lista', [1,7], 9, 8, 7]

- ***** : Para repetir las listas

```
lis2 = lis1 * 2
```

lis2 quedaría:[22, True, 'una lista', [1,7], 22, True, 'una lista', [1,7]]



Operadores

```
lista1 = [[1, 2]] * 3
```

```
lista2 = [[1, 2], [1, 2], [1, 2]]
```

```
lista3 = [1, 2] * 3
```

```
lista1[0][1] = 'cambio'
```

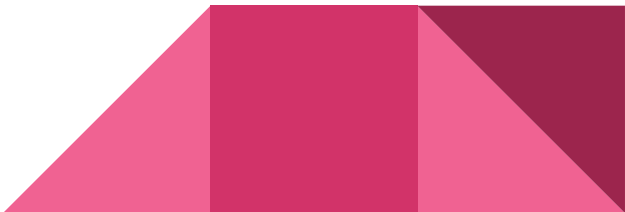
```
lista2[0][1] = 'cambio'
```

```
lista3[1] = 'cambio'
```

```
print(lista1)      # [[1, 'cambio'], [1, 'cambio'], [1, 'cambio']]
```

```
print(lista2)      # [[1, 'cambio'], [1, 2], [1, 2]]
```

```
print(lista3)      # [1, 'cambio', 1, 2, 1, 2]
```



Recorrer una lista

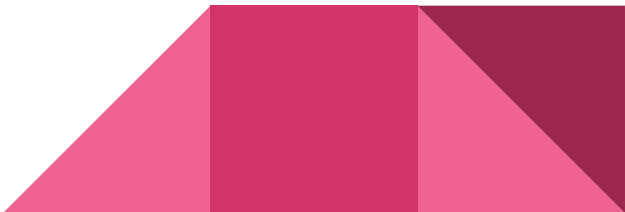
```
lista = [ 22, True, 'una lista', [1,7] ]
```

```
for elem in lista:
```

```
    print(elem)
```

```
for indice in range(len(lista)):
```

```
    print(lista[indice])
```



Lista por comprensión

```
cuadrados1 = [num**2 for num in range(10)]
```

```
print(cuadrados) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
cuadrados2 = [num**2 for num in range(10) if num % 2 == 0]
```

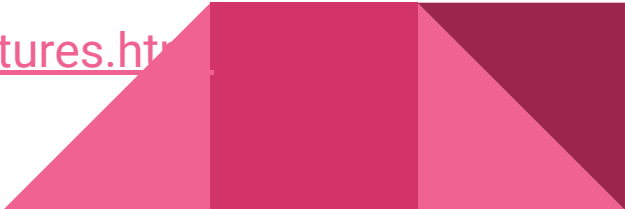
```
print(cuadrados2) # [0, 4, 16, 36, 64]
```



Algunas funciones de listas

- `lista.copy()` # Devuelve una copia en otra con otro id
- `lista.index(elem)` # Retorna el índice de elem
- `lista.append(elem)` # Agrega elem al final de la lista
- `lista.remove(elem)` # Quita el primer ítem de la lista cuyo valor sea elem
- `lista.count(elem)` # Retorna la cantidad de veces que elem aparece en la lista
- `lista.sort()` # Ordena los elementos de la lista
- `lista.pop([i])` # Quita y devuelve el elemento de la posición i (opcional)
- `lista.insert(i, elem)` # Inserta un elemento en la posición i

Más info <https://docs.python.org/es/3/tutorial/datastructures.html>



Ejercicio 1

Escribir un programa que le pida al usuario ingresar asignaturas de un curso (por ejemplo Matemáticas, Física, Química, Historia y Lengua) hasta que se ingrese “-1” y guardarlos en una lista. Luego muestre por pantalla el mensaje Yo estudio <asignatura>, donde <asignatura> es cada una de las asignaturas de la lista por línea.



Ejercicio 2

Escribir un programa que pregunte por una muestra de números, separados por comas, los guarde en una lista y muestre por pantalla su promedio.

hint: `sum(lista)`



Ejercicio 3

Escribir un programa que pregunte al usuario los 7 números ganadores de la lotería, los almacene en una lista y los muestre por pantalla ordenados de menor a mayor separados por un guión.

hint: `str.split()`



Ejercicio 4

Escribir un programa que almacene el abecedario en una lista, elimine de la lista las letras que ocupen posiciones múltiplos de 3, y muestre por pantalla la lista resultante. Usar el módulo `string.ascii_uppercase`

resultado: ['A', 'B', 'D', 'E', 'G', 'H', 'J', 'K', 'M', 'N', 'P', 'Q', 'S', 'T', 'V', 'W', 'Y', 'Z']



Ejercicio 5

Escribir un programa que tenga ya almacenado las asignaturas de un curso (por ejemplo Matemáticas, Física, Química, Historia y Lengua) en una lista, pregunte al usuario la nota que ha sacado en cada asignatura. Se deberá imprimir por consola el promedio y la cantidad de notas que se encuentran por debajo de este.



Ejercicio 6

Escribir un programa que tenga almacenado las asignaturas de un curso (por ejemplo Matemáticas, Física, Química, Historia y Lengua) en una lista, pregunte al usuario la nota que ha sacado en cada asignatura y elimine de la lista las asignaturas aprobadas. Al final el programa debe mostrar por pantalla las asignaturas que el usuario tiene que repetir. (Se aprueba con 7 o más).

